

ShopVerse

Infrastructure Services

[Use Cases](#) • [Logging](#) • [Value Inspection](#)

A hands-on reference for developers running PostgreSQL, Redis, Cassandra, MongoDB, Neo4j, Elasticsearch, Zookeeper & Kafka locally via Docker — with every command you need to debug them.

[PostgreSQL](#)[Redis](#)[Cassandra](#)[MongoDB](#)[Neo4j](#)[Elasticsearch](#)[Zookeeper](#)[Kafka](#)

Contents

Service	Role in ShopVerse	Page
PostgreSQL	Primary relational store — customers, orders, products	2
Redis	Cache, sessions, Pub/Sub, distributed locks	3
Cassandra	Wide-column store — order activity time-series	4
MongoDB	Document store — product reviews & rich documents	5
Neo4j	Graph DB — product recommendations	6
Elasticsearch	Full-text & faceted product search	7
Zookeeper	Kafka coordinator & distributed config	8
Kafka	Async event streaming — order events, notifications	9

PostgreSQL — Primary relational database

Use Cases in ShopVerse

- Stores customers, products, orders and order_items — the core transactional data
- Enforces referential integrity via FK constraints (orders -> customers, order_items -> products)
- Optimistic locking via version column prevents lost updates under concurrency
- Flyway migrations (V1, V2, V3...) manage schema evolution without downtime
- GIN trigram index on products.name enables fast LIKE/ILIKE product search
- Audit trigger (trg_products_updated_at) auto-updates updated_at on every write
- HikariCP connection pool (max 20) manages connection reuse for throughput
- Used as the WRITE store in CQRS — commands go to Postgres, reads may go to cache/ES

How ShopVerse Uses It

Every REST call to POST /api/orders writes to PostgreSQL via JPA repositories.

ProductJpaRepository.findByCategory() issues a SELECT with index scan on idx_products_category.

Flyway runs on startup (baseline-on-migrate=true) to apply any pending migrations automatically.

Getting Logs

Command	What it shows
<code>docker logs shopverse-postgres</code>	All container logs
<code>docker logs shopverse-postgres --tail 100 -f</code>	Follow last 100 lines live
<code>docker logs shopverse-postgres 2>&1 grep ERROR</code>	Filter for errors only
<code>docker logs shopverse-postgres 2>&1 grep FATAL</code>	Fatal startup errors
<code>docker exec shopverse-postgres tail -f /var/log/postgresql/postgresql*.log</code>	PG log file (if configured)
In <code>application.yml</code> : <code>org.hibernate.SQL: DEBUG</code>	Prints every SQL query Spring issues
In <code>application.yml</code> : <code>org.hibernate.type.descriptor.sql: TRACE</code>	Prints bind parameter values too

Inspecting Values

Command	What it shows
<code>docker exec -it shopverse-postgres psql -U shopverse</code>	Open interactive psql shell
<code>\l</code>	List all databases
<code>\c shopverse</code>	Connect to shopverse DB
<code>\dt</code>	List all tables
<code>\d customers</code>	Describe customers table schema

<code>SELECT * FROM customers LIMIT 5;</code>	View first 5 customers
<code>SELECT * FROM products WHERE active=true LIMIT 10;</code>	Active products
<code>SELECT * FROM orders ORDER BY created_at DESC LIMIT 5;</code>	Latest 5 orders
<code>SELECT * FROM flyway_schema_history;</code>	View migration history
<code>EXPLAIN ANALYZE SELECT * FROM products WHERE name ILIKE '%phone%';</code>	Query execution plan
<code>SELECT schemaname,tablename,attname,n_distinct,correlation FROM pg_stats WHERE tablename='products';</code>	Column statistics

Pro Tips

- ✓ Add `?loggerLevel=DEBUG` to JDBC URL to log all SQL with bind params without Hibernate config
- ✓ Run `VACUUM ANALYZE products;` after bulk inserts to update planner statistics
- ✓ `pg_stat_statements` extension tracks slow queries — enable with `CREATE EXTENSION pg_stat_statements`
- ✓ Use `EXPLAIN (ANALYZE, BUFFERS)` to see cache hit ratio alongside execution plan

Redis — Cache, Pub/Sub, distributed locks, sessions

Use Cases in ShopVerse

- Product cache — GET /api/products hits Redis first (Caffeine L1 + Redis L2)
- Session/token cache — JWT blacklist and invalidated tokens stored with TTL
- Distributed locks via Redisson RLock — prevents duplicate order processing
- Idempotency keys — stores processed request IDs (TTL 24h) to deduplicate retries
- Pub/Sub messaging — order events published to 'order-events' channel for real-time updates
- Rate limiting counters — sliding window counters per user/IP
- Shopping cart state — transient cart data stored as Redis hashes per user
- Leaderboard / loyalty points ranking via Redis sorted sets

How ShopVerse Uses It

ProductCacheService uses @Cacheable('products') — Spring Cache stores results in Redis.

RedisPubSubPublisher.publish('order-events', event) fires on every order state change.

RedissonConfig provides a custom RLock bean used in OrderService.placeOrder() for idempotency.

Getting Logs

Command	What it shows
<code>docker logs shopverse-redis</code>	All Redis container logs
<code>docker logs shopverse-redis --tail 50 -f</code>	Follow last 50 lines
<code>docker exec shopverse-redis redis-cli MONITOR</code>	Live stream of ALL commands (dev only!)
<code>docker exec shopverse-redis redis-cli SLOWLOG GET 10</code>	Last 10 slow commands (>10ms)
<code>docker exec shopverse-redis redis-cli INFO server</code>	Server version and uptime
<code>docker exec shopverse-redis redis-cli INFO stats</code>	Ops/sec, keyspace hits/misses
<code>docker exec shopverse-redis redis-cli INFO memory</code>	Memory usage and fragmentation

Inspecting Values

Command	What it shows
<code>docker exec -it shopverse-redis redis-cli</code>	Open interactive Redis CLI
<code>PING</code>	Check connectivity (returns PONG)
<code>DBSIZE</code>	Total number of keys
<code>KEYS *</code>	All keys (avoid in production)
<code>KEYS products:*</code>	Keys matching pattern

SCAN 0 MATCH products:* COUNT 20	Safe paginated key scan
TYPE products::SimpleKey []	Data type of a key
GET products::SimpleKey []	Value of a string/cache key
TTL products::SimpleKey []	Remaining TTL in seconds
HGETALL cart:user:42	All fields of a hash
LRANGE order-events 0 -1	All items in a list
ZRANGE leaderboard 0 9 WITHSCORES	Top 10 sorted set entries
SUBSCRIBE order-events	Subscribe to a Pub/Sub channel
INFO keyspace	Keys per database with TTL counts
MEMORY USAGE products::SimpleKey []	Bytes used by one key

Pro Tips

- ✓ Use SCAN instead of KEYS in production — KEYS blocks the event loop
- ✓ redis-cli --stat gives a live refreshing stats summary (like top for Redis)
- ✓ Set maxmemory-policy allkeys-lru so Redis evicts LRU keys when memory is full
- ✓ OBJECT ENCODING key shows compact encoding (ziplist vs hashtable) for memory tuning

Cassandra — Wide-column store — time-series event log

Use Cases in ShopVerse

- `order_activity` table stores every order event per customer ordered by time (DESC)
- Partition key = `customer_id` — all activity for one customer lives on the same node
- Clustering columns = `event_time` + `event_id` — enables time-range queries efficiently
- Write-optimised — handles millions of append-only event writes with low latency
- Ideal for audit trails, activity feeds, IoT sensor data, and time-series analytics
- No JOINS — data is denormalised by query pattern (query-driven data modelling)
- Tunable consistency — use ONE for fast writes, QUORUM for reads requiring accuracy
- Compaction strategies (TWCS) optimise for time-series data with automatic TTL cleanup

How ShopVerse Uses It

`OrderActivityRepository.save(activity)` writes to Cassandra every time an order transitions state.

`CustomerActivityQuery.getRecentActivity(customerId, limit)` reads the last N events for a customer.

`CassandraConfig` bootstraps the keyspace and `order_activity` table on first startup.

Getting Logs

Command	What it shows
<code>docker logs shopverse-cassandra</code>	All Cassandra container logs
<code>docker logs shopverse-cassandra --tail 100 -f</code>	Follow last 100 lines
<code>docker logs shopverse-cassandra 2>&1 grep ERROR</code>	Errors only
<code>docker logs shopverse-cassandra 2>&1 grep WARN</code>	Warnings (compaction, GC, etc.)
<code>docker exec shopverse-cassandra tail -f /var/log/cassandra/system.log</code>	Cassandra system log file
<code>docker exec shopverse-cassandra nodetool tpstats</code>	Thread pool stats — queue depths
<code>docker exec shopverse-cassandra nodetool gcstats</code>	Garbage collection pause times

Inspecting Values

Command	What it shows
<code>docker exec -it shopverse-cassandra cqlsh</code>	Open CQL shell
<code>DESCRIBE KEYSPACES;</code>	List all keyspaces
<code>USE shopverse;</code>	Switch to shopverse keyspace
<code>DESCRIBE TABLES;</code>	List tables in current keyspace

<code>DESCRIBE TABLE order_activity;</code>	Full schema with partition/clustering keys
<code>SELECT * FROM order_activity LIMIT 10;</code>	First 10 rows
<code>SELECT * FROM order_activity WHERE customer_id=1 LIMIT 5;</code>	Activity for customer 1
<code>SELECT * FROM order_activity WHERE customer_id=1 AND event_time > '2024-01-01' LIMIT 10;</code>	Time-range query for a customer
<code>SELECT COUNT(*) FROM order_activity;</code>	Total row count (slow on large tables)
<code>docker exec shopverse-cassandra nodetool status</code>	Ring status — all nodes UP/DOWN
<code>docker exec shopverse-cassandra nodetool tablestats shopverse.order_activity</code>	Table stats: read/write latency, size
<code>docker exec shopverse-cassandra nodetool cfhistograms shopverse order_activity</code>	Read/write latency histogram

Pro Tips

- ✓ nodetool status should show UN (Up/Normal) — UJ means still joining ring, wait before writing
- ✓ Always filter by partition key (customer_id) first — full table scans are very slow
- ✓ Use TRACING ON in cqlsh to see exactly which nodes served the query
- ✓ TTL on rows: `INSERT ... USING TTL 2592000` auto-deletes data after 30 days

MongoDB — Document store — reviews & rich content

Use Cases in ShopVerse

- Stores product reviews — flexible schema accommodates any review structure (ratings, media, tags)
- Rich product metadata — technical specs, attributes, variants stored as nested documents
- User activity documents — browsing history, wishlist, preference profiles
- Content management — blog posts, banners, promotional content with varied fields
- Aggregation pipelines compute average ratings, top reviewers, sentiment scores
- Atlas Search / text indexes enable full-text review search and keyword filtering
- \$lookup joins review documents with product data for enriched responses
- Change streams enable real-time notifications when new reviews are posted

How ShopVerse Uses It

ReviewRepository (MongoRepository) persists ProductReview documents with customerId, productId, rating.

ReviewService.getAverageRating(productId) runs an aggregation: group by productId, avg(rating).

MongoDB auto-creates the shopverse database and reviews collection on first insert.

Getting Logs

Command	What it shows
<code>docker logs shopverse-mongo</code>	All MongoDB container logs
<code>docker logs shopverse-mongo --tail 50 -f</code>	Follow last 50 lines
<code>docker logs shopverse-mongo 2>&1 grep -i error</code>	Error lines
<code>docker exec -it shopverse-mongo mongosh --eval "db.setProfilingLevel(1, {slowms: 100})"</code>	Enable slow query profiler (>100ms)
<code>docker exec -it shopverse-mongo mongosh shopverse --eval "db.system.profile.find().limit(5).sort({ts:-1}).pretty()"</code>	Last 5 slow queries
<code>docker exec -it shopverse-mongo mongosh --eval "db.adminCommand({getLog:'global'})"</code>	Global server log

Inspecting Values

Command	What it shows
<code>docker exec -it shopverse-mongo mongosh</code>	Open MongoDB shell
<code>show dbs</code>	List all databases
<code>use shopverse</code>	Switch to shopverse database
<code>show collections</code>	List all collections
<code>db.reviews.find().limit(5).pretty()</code>	First 5 review documents

<code>db.reviews.findOne({productId: 1})</code>	One review for product 1
<code>db.reviews.countDocuments()</code>	Total document count
<code>db.reviews.find({rating: {\$gte: 4}}).count()</code>	Reviews with 4+ stars
<code>db.reviews.aggregate([{\$group: {_id: '\$productId', avg: {\$avg: '\$rating'}}}])</code>	Average rating per product
<code>db.reviews.getIndexes()</code>	List all indexes on reviews
<code>db.stats()</code>	Database stats: size, collections
<code>db.reviews.stats()</code>	Collection stats: size, index size
<code>db.currentOp()</code>	Currently running operations

Pro Tips

- ✓ Use `.explain('executionStats')` to verify queries use indexes: `db.reviews.find({productId:1}).explain()`
- ✓ `db.reviews.createIndex({productId:1, rating:-1})` speeds up per-product rating queries
- ✓ mongosh supports tab-completion — type `db.reviews.` and press Tab to see all methods
- ✓ Compass GUI (free) gives visual document browser, query builder and explain viewer

Neo4j — Graph database — product recommendations

Use Cases in ShopVerse

- Models products as nodes and co-purchase/co-view relationships as edges
- Recommendation query: find products related to X within 2 hops of the graph
- Customer-product PURCHASED relationships enable collaborative filtering
- Category hierarchy modelled as IS_A graph — traverse subcategories in one query
- Fraud detection: find circular transaction patterns via graph traversal
- Shortest path between products reveals hidden associations for cross-selling
- Graph algorithms (PageRank, Louvain community) rank and cluster products
- Similar customers: match customers sharing 3+ purchased products

How ShopVerse Uses It

ProductNode entity (@Node) and SIMILAR_TO relationship stored in Neo4j.

ProductGraphRepository.findRecommendations(productId) runs Cypher query for similar products.

GET /api/graphql productRecommendations(productId) calls this repository.

Neo4j credentials: neo4j / shopverse (set via NEO4J_AUTH in docker-compose).

Getting Logs

Command	What it shows
<code>docker logs shopverse-neo4j</code>	All Neo4j container logs
<code>docker logs shopverse-neo4j --tail 100 -f</code>	Follow last 100 lines
<code>docker logs shopverse-neo4j 2>&1 grep ERROR</code>	Errors only
<code>docker exec shopverse-neo4j tail -f /logs/neo4j.log</code>	Neo4j application log
<code>docker exec shopverse-neo4j tail -f /logs/query.log</code>	Slow query log (if enabled)
<code>http://localhost:7474 -> :sysinfo</code>	Browser UI: system info & metrics

Inspecting Values

Command	What it shows
<code>http://localhost:7474</code>	Neo4j Browser UI (user: neo4j, pw: shopverse)
<code>docker exec -it shopverse-neo4j cypher-shell -u neo4j -p shopverse</code>	Cypher shell in terminal
<code>SHOW DATABASES;</code>	List all databases
<code>MATCH (n) RETURN count(n);</code>	Total node count
<code>MATCH (n:Product) RETURN n LIMIT 5;</code>	First 5 Product nodes

<code>MATCH (a:Product)-[r]->(b:Product) RETURN a,r,b LIMIT 10;</code>	Product relationships
<code>MATCH (p:Product {productId: 1})-[:SIMILAR_TO]-(rec) RETURN rec;</code>	Recommendations for product 1
<code>CALL db.schema.visualization()</code>	Visual schema of node/relationship types
<code>CALL db.indexes()</code>	List all indexes
<code>PROFILE MATCH (p:Product)-[:SIMILAR_TO]-(r) WHERE p.productId=1 RETURN r</code>	Query execution plan
<code>CALL dbms.listQueries()</code>	Currently running queries
<code>CALL dbms.queryJmx('org.neo4j:instance=kernel#0, name=Store file sizes')</code>	Store sizes on disk

Pro Tips

- ✓ Neo4j Browser (localhost:7474) renders graph visuals — great for exploring relationships
- ✓ Use EXPLAIN before PROFILE to get plan without running query (faster for schema checks)
- ✓ Create indexes: CREATE INDEX FOR (p:Product) ON (p.productId) for faster lookups
- ✓ CALL apoc.meta.stats() from APOC plugin gives schema and relationship count overview

Elasticsearch — Full-text and faceted product search

Use Cases in ShopVerse

- Full-text search on product name, description with relevance scoring (BM25)
- Autocomplete / search-as-you-type via edge_ngram analyzer on product names
- Faceted filtering — filter by category, price range, in-stock simultaneously
- Fuzzy matching — finds 'iphon' when user types 'iphone' (edit distance = 1)
- Boost rules — promoted products or high-rated items rank higher in results
- Aggregations — compute min/max price, category counts for filter UI
- Scroll / search_after for paginating through thousands of results efficiently
- Index sync from PostgreSQL via Elasticsearch sync service on every product write

How ShopVerse Uses It

ElasticsearchProductRepository indexes products under 'products' index.

SearchProductsUseCase queries ES with multi_match across name and description fields.

GET /api/search?keyword=phone&category=Electronics routes to ES query.

Product writes trigger ElasticSyncService.sync(product) to keep index fresh.

Getting Logs

Command	What it shows
<code>docker logs shopverse-elasticsearch</code>	All ES container logs
<code>docker logs shopverse-elasticsearch --tail 100 -f</code>	Follow last 100 lines
<code>docker logs shopverse-elasticsearch 2>&1 grep WARN</code>	Warnings (GC, disk, etc.)
<code>curl -s http://localhost:9200/_cat/nodes?v</code>	Node health summary
<code>curl -s http://localhost:9200/_cluster/health?pretty</code>	Cluster health: green/yellow/red
<code>curl -s 'http://localhost:9200/_cat/indices?v&s=index'</code>	All indexes with doc count and size
<code>curl -s http://localhost:9200/_nodes/stats?pretty python3 -m json.tool grep heap</code>	JVM heap usage per node

Inspecting Values

Command	What it shows
<code>curl -s http://localhost:9200</code>	Version and cluster name
<code>curl -s http://localhost:9200/_cat/indices?v</code>	All indexes, doc counts, health

<code>curl -s http://localhost:9200/products/_mapping?pretty</code>	Field mappings of products index
<code>curl -s http://localhost:9200/products/_count</code>	Total documents in products index
<code>curl -s 'http://localhost:9200/products/_search?pretty&size;=3'</code>	First 3 documents
<code>curl -s -X POST 'http://localhost:9200/products/_search?pretty' -H 'Content-Type: application/json' -d '{"query":{"match":{"name":"phone"}}}'</code>	Search for 'phone' in name
<code>curl -s http://localhost:9200/products/_stats?pretty</code>	Index stats: docs, store size, search
<code>curl -s 'http://localhost:9200/_cat/shards?v&s;=index'</code>	Shard allocation across nodes
<code>curl -s http://localhost:9200/_tasks?detailed=true&pretty;</code>	Currently running tasks
<code>curl -s http://localhost:9200/_nodes/hot_threads</code>	Hot threads on node (CPU debugging)

Pro Tips

- ✓ Install Kibana (port 5601) alongside ES for a visual query builder and index browser
- ✓ Use `_explain` API to see why a document did or did not match: `POST /products/_explain/{id}`
- ✓ Slow log threshold: `PUT /products/_settings {"index.search.slowlog.threshold.query.warn":"1s"}`
- ✓ Run `curl http://localhost:9200/_cat/health?v` every few seconds during indexing to watch progress

Zookeeper — Kafka coordinator & distributed config

Use Cases in ShopVerse

- Kafka broker metadata: tracks which brokers are alive, leader elections for partitions
- Topic configuration: stores partition counts, replication factor, retention policy
- Consumer group coordination: stores committed offsets before Kafka 2.x (now optional)
- Controller election: one Kafka broker is elected 'controller' via ZK leader election
- Distributed lock: ZK znodes can be used as distributed locks across services
- Service discovery: register service endpoints as ephemeral znodes (alive = present)
- Configuration management: store shared config that multiple services read
- Cluster membership: track which nodes in a distributed system are currently active

How ShopVerse Uses It

ShopVerse uses Confluent Kafka 7.6 with traditional ZooKeeper mode (KRaft not yet enabled).

ZooKeeper runs on port 2181, internal to the shopverse-net Docker network.

Kafka broker registers itself at startup and ZK manages broker-to-partition assignments.

You rarely interact with ZK directly — Kafka CLI tools handle ZK coordination for you.

Getting Logs

Command	What it shows
<code>docker logs shopverse-zookeeper</code>	All ZooKeeper container logs
<code>docker logs shopverse-zookeeper --tail 50 -f</code>	Follow last 50 lines
<code>docker logs shopverse-zookeeper 2>&1 grep -i error</code>	Errors only
<code>docker logs shopverse-zookeeper 2>&1 grep -i exception</code>	Java exceptions
<code>docker exec shopverse-zookeeper bash -c "echo ruok nc localhost 2181"</code>	4-letter word check: returns 'imok'
<code>docker exec shopverse-zookeeper bash -c "echo stat nc localhost 2181"</code>	Server stats: connections, latency
<code>docker exec shopverse-zookeeper bash -c "echo mntr nc localhost 2181"</code>	Full metrics dump

Inspecting Values

Command	What it shows
<code>docker exec shopverse-zookeeper bash -c "echo ruok nc localhost 2181"</code>	Health check — 'imok' means healthy
<code>docker exec shopverse-zookeeper bash -c "echo srvr nc localhost 2181"</code>	Server role: leader or follower

<code>docker exec shopverse-zookeeper bash -c "echo cons nc localhost 2181"</code>	Active client connections
<code>docker exec shopverse-zookeeper bash -c "echo dump nc localhost 2181"</code>	Outstanding sessions and ephemeral nodes
<code>docker exec -it shopverse-zookeeper /bin/zookeeper-shell.sh localhost:2181</code>	ZK shell (interactive CLI)
<code>ls /brokers/ids</code>	In ZK shell: list registered Kafka brokers
<code>ls /brokers/topics</code>	In ZK shell: all Kafka topics
<code>get /brokers/ids/1</code>	In ZK shell: broker 1 metadata JSON
<code>ls /consumers</code>	In ZK shell: old-style consumer groups

Pro Tips

- ✓ `echo ruok | nc localhost 2181` is the quickest ZK health check — 'imok' = all good
- ✓ 4-letter words (ruok, stat, mntr, cons) must be whitelisted in `zoo.cfg` for ZK 3.5+
- ✓ In Kafka 3.x with KRaft mode, ZooKeeper is no longer needed — Confluent is migrating
- ✓ ZK should never be the bottleneck — if latency spikes, check JVM GC via `mntr` output

Kafka — Async event streaming — orders & notifications

Use Cases in ShopVerse

- Order events published to 'order-events' topic when orders are created/updated
- Notification service consumes order events to send email/push confirmations
- Inventory service consumes order.placed events to deduct stock asynchronously
- Analytics pipeline consumes all events for real-time dashboards and reporting
- Dead Letter Queue (DLQ) — failed messages routed to order-events.DLT topic for retry
- Exactly-once semantics with idempotent producer (acks=all, retries=3)
- Replay capability: rewind consumer offset to re-process historical events
- Kafka Streams for real-time aggregations: order totals per customer per hour

How ShopVerse Uses It

OrderEventProducer.publish(event) sends OrderPlacedEvent to 'order-events' topic.

OrderEventConsumer @KafkaListener(topics='order-events') processes incoming events.

Bootstrap servers: localhost:9092 (EXTERNAL listener for host-side Spring Boot app).

Internal Docker services use kafka:29092 (INTERNAL listener).

Getting Logs

Command	What it shows
<code>docker logs shopverse-kafka</code>	All Kafka container logs
<code>docker logs shopverse-kafka --tail 100 -f</code>	Follow last 100 lines live
<code>docker logs shopverse-kafka 2>&1 grep ERROR</code>	Error lines only
<code>docker logs shopverse-kafka 2>&1 grep 'Leader election'</code>	Partition leader elections
<code>docker logs shopverse-kafka 2>&1 grep 'Created topic'</code>	Topic creation events
In application.yml: <code>org.springframework.kafka: DEBUG</code>	Spring Kafka producer/consumer debug
In application.yml: <code>org.apache.kafka: INFO</code>	Kafka client library info logs

Inspecting Values

Command	What it shows
<code>docker exec shopverse-kafka kafka-topics --bootstrap-server localhost:9092 --list</code>	List all topics

<code>docker exec shopverse-kafka kafka-topics --bootstrap-server localhost:9092 --describe --topic order-events</code>	Topic details: partitions, replicas, ISR
<code>docker exec shopverse-kafka kafka-topics --bootstrap-server localhost:9092 --create --topic order-events --partitions 3 --replication-factor 1</code>	Create a topic manually
<code>docker exec -it shopverse-kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic order-events --from-beginning</code>	Read all messages from beginning
<code>docker exec -it shopverse-kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic order-events</code>	Read new messages only (live tail)
<code>docker exec -it shopverse-kafka kafka-console-producer --bootstrap-server localhost:9092 --topic order-events</code>	Publish test messages interactively
<code>docker exec shopverse-kafka kafka-consumer-groups --bootstrap-server localhost:9092 --list</code>	List all consumer groups
<code>docker exec shopverse-kafka kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group shopverse-consumer</code>	Consumer group lag per partition
<code>docker exec shopverse-kafka kafka-log-dirs --bootstrap-server localhost:9092 --topic-list order-events --describe</code>	Log dirs and offsets on disk
<code>docker exec shopverse-kafka kafka-get-offsets --bootstrap-server localhost:9092 --topic order-events</code>	Latest offset per partition

Pro Tips

- ✓ Consumer LAG = latest offset - committed offset; high lag means consumers are falling behind
- ✓ Use `--from-beginning` to replay all messages; omit it to tail only new messages
- ✓ `kafka-consumer-groups --reset-offsets --to-earliest` lets you replay events for a group
- ✓ Kafka UI (github.com/provectus/kafka-ui) is a free Docker container for visual topic browsing

Quick Reference Cheatsheet

Health Check — One-Liners

Command	Service
<code>docker exec shopverse-postgres psql -U shopverse -c 'SELECT 1'</code>	PostgreSQL
<code>docker exec shopverse-redis redis-cli PING</code>	Redis
<code>docker exec shopverse-cassandra nodetool status</code>	Cassandra
<code>docker exec shopverse-mongo mongosh --eval 'db.runCommand({ping:1})'</code>	MongoDB
<code>curl -s http://localhost:7474</code>	Neo4j
<code>curl -s http://localhost:9200/_cluster/health</code>	Elasticsearch
<code>echo ruok nc localhost 2181</code>	ZooKeeper
<code>docker exec shopverse-kafka kafka-topics --bootstrap-server localhost:9092 --list</code>	Kafka

All Logs at Once

```
# Tail logs of all services simultaneously
docker compose logs -f --tail=50

# Tail specific services
docker compose logs -f postgres redis kafka

# Grep errors across all services
docker compose logs 2>&1 | grep -i error
```

Data Type Summary

Service	Data Type	ShopVerse Data	Port
PostgreSQL	Relational (ACID)	customers, products, orders	5432
Redis	Key-Value / Structures	cache, locks, sessions, pub/sub	6379
Cassandra	Wide-Column	order_activity (time-series)	9042
MongoDB	Document (BSON)	reviews, rich product metadata	27017
Neo4j	Graph	product relationships, recs	7687
Elasticsearch	Inverted Index	product search index	9200

Zookeeper	Coordination	Kafka metadata / leader election	2181
Kafka	Append-Log / Streams	order-events topic	9092

Tip: Run `docker compose ps` to see all container statuses at a glance. Green dot = running. Use `docker stats` to see live CPU/memory usage per container.